

SemPointer: Toward Random-Access Semantic Memory for Large Language Models

Viksit Garg¹

Department of Artificial Intelligence & Machine Learning, Dr. Akhilesh Das Gupta
Institute of Professional Studies, New Delhi, India
viksitg427@gmail.com

Abstract. Full-context inference feeds every token to attention, so cost grows steeply with context length. SemPointer instead stores context blocks in an off-attention text substrate and keeps short pointer strings in the active prompt; per query, an off-the-shelf dense encoder (all-MiniLM-L6-v2) selects blocks by cosine similarity and resolves them to verbatim text. The pointer strings themselves do not affect selection. We measured Qwen2.5-3B-Instruct (NF4, greedy decoding, seed 42) on RULER containment (n=180), LongBench F1 (n=1150), LoCoMo under a token-F1 adaptation (n=199), and a KV-compress pilot (n=10). Against full context, SemPointer uses 10–15× fewer active tokens but lowers quality: RULER -0.37 (95% CI $[-0.44, -0.30]$), LongBench -0.033 ($[-0.040, -0.027]$), LoCoMo -0.012 ($[-0.022, -0.003]$, not significant after Bonferroni correction). Against dense RAG it ties on quality at higher token cost. Scope: one model, one seed, text substrate only, contexts capped at 16k tokens.

Keywords: long-context inference · dense retrieval · prompt compression · benchmarking · conversational memory

1 Introduction

1.1 Motivation and Problem Setting

Transformer models [29] process textual context as an unbroken linear sequence. In multi-turn dialogue, long-horizon programming, and iterative document synthesis, historical context accumulates across successive inference calls. The computational and memory burden of this accumulated history depends fundamentally on the deployment regime:

Regime 1 (Stateless / no-cache inference): When no persistent KV cache is maintained—such as in stateless API endpoints, cross-session interactions, or when contexts exceed GPU memory—each request pays a prefill pass over the full prefix history. Prefill attention computation scales as $\mathcal{O}(L^2d)$ per request [29], producing cumulative cost $\sum_{t=1}^T \mathcal{O}(L_t^2)$ across T turns; per-token decoding with a KV cache costs $\mathcal{O}(Ld)$. Eq. (7) and the FLOP cost equations (Section 6) model per-request prefill cost, not per-step decode cost.

Regime 2 (Persistent KV-cache inference): In warm, resident GPU caches [12], recompute FLOPs approach zero. The binding constraint shifts from arithmetic compute to GPU memory capacity; once full, eviction or secondary storage offload is required.

Regime 3 (Paged / host-offloaded KV): Systems like PagedAttention [12] manage non-contiguous KV blocks between GPU and host memory. Here, host-device bandwidth and paging latency become the primary system bottlenecks.

Context compression methods—hard token pruning [8, 9, 23], learned continuous soft prompts [3, 6, 15, 20], and dynamic KV cache eviction [10, 17]—address Regime 1 FLOPs and mitigate memory pressure in Regimes 2–3. However, they share a structural limitation: they reduce context representations in place, without establishing an addressable indirection layer. Once a prompt is compressed into summary vectors or pruned text, the model processes that proxy uniformly across all downstream turns, regardless of whether specific details within that block relate to the immediate query. SemPointer addresses this gap by providing explicit semantic addresses into an independent substrate, yielding FLOP savings in Regime 1 and memory footprint reduction in Regimes 2–3.

1.2 The Indirection Hypothesis

Operating systems separate memory addresses from stored payloads, passing compact references through registers and deferring payload retrieval until execution. We hypothesize that an analogous indirection principle can be realized within the sequence space of LLMs:

***The Indirection Hypothesis:** By generating compact, token-level semantic pointers that reference independently stored contextual substrates, an LLM architecture can support selective, address-based access to historical context, compressing the active sequence from approximately $N \cdot B$ to $(N - m)k + m \cdot B_{\text{res}}$ tokens per query ($k \ll B, m \ll N$) by bypassing the ingestion of unselected context payloads.*

1.3 Research Questions

This study investigates three primary theoretical and empirical questions:

- *RQ1 (Addressing Feasibility):* Can a compact sequence of discrete or soft tokens function as a stable, unique semantic address for independently stored context?
- *RQ2 (Resolution Fidelity):* Can pointer resolution reconstruct task-relevant features with fidelity comparable to full-context exposure?
- *RQ3 (Amortized Complexity):* Under what access distributions and query volumes does pointer indirection yield computational savings over repeated linear re-ingestion?

Table 1. Comparison of Context Management Strategies in Large Language Models

Paradigm	Mechanism	Stores Original?	Explicit Indirection?	Selective Access?	Reusability	Active Sequence / Complexity Mitigation
Full Linear Ingestion [29]	Dense Attention	Yes (In Context)	No	No	Inefficient	None ($\mathcal{O}(L^2)$)
Hard Token Pruning [8]	Token Elimination	No	No	No	No	Moderate (Length Pruned)
Soft Prompt Compression [6, 20]	Learned Latent Tokens	No	No	No	Yes	Moderate (Prefix Condensed)
Dynamic Cache Eviction [10]	KV Discard/Merge	Partial	No	No	No	High (Memory Capped)
Landmark Attention [19]	Gated Attention Routing	Yes (In Context)	No (Intra-pass)	Yes (Blocks)	No	High (Within single pass)
Retrieval-Augmented Gen. [2, 14]	External Vector DB	Yes (External)	Partial (Dense Vector)	Yes	Yes	High (Only top- k retrieved)
SemPointer (Proposed)	Semantic Indirection	Yes (Substrate)	Yes (Token Address)	Yes (Query-Cond.)	Yes	Compressed ($\mathcal{O}((Nk + mB_{\text{max}} + L_Q)^2)$ vs. $\mathcal{O}((NB + L_Q)^2)$)

1.4 Contributions

This paper presents six contributions uniting formal modeling with rigorous empirical verification:

1. We distinguish context compression from semantic addressing, establishing seven axiomatic criteria required for true pointer mechanics.
2. We formalize the complete SemPointer lifecycle across Encounter, Storage, Address Generation, Selection, and Resolution (Fig. 1).
3. Under assumptions A1–A5 and the stateless-inference cost model (Section 6), we derive the algebraic crossover threshold K^* and single-query viability condition ($N \geq 2$) for active-sequence compression.
4. We characterize three primary theoretical failure modes: address collisions, positional semantic drift, and resolution divergence.
5. We contrast SemPointer with Landmark Attention [19], RAG [2, 14], MemGPT [22], and cognitive SPA [4], providing a structured feature matrix (Table 3).
6. We formulate six falsifiable hypotheses (H_1 – H_6) and test them on executed benchmark runs (RULER $n=180$, LongBench $n=1150$, LoCoMo $n=199$, KV pilot $n=10$, multi-hop pilot $n=30$); K^* crossover figures are analytic derivations, not measurements.

2 Background and Prior-Art Spectrum

2.1 Dense Autoregressive Attention and Scaling Walls

In standard autoregressive generation [29], given an input sequence $X = (x_1, x_2, \dots, x_L)$, each self-attention layer computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

where $Q, K, V \in \mathbb{R}^{L \times d}$. Computing QK^T costs $\mathcal{O}(L^2d)$ with $\mathcal{O}(2n_{\text{layers}}n_{\text{heads}}Ld_k)$ KV state. Across T turns with $L_t = \sum_{i=1}^t |X_i|$, cumulative cost scales as $\sum_{t=1}^T \mathcal{O}(L_t^2)$. Table 1 positions SemPointer among context-management paradigms.

2.2 Hard Prompt Compression

Hard prompt compression (e.g., LLMingua [8], LongLLMingua [9], and LLMingua-2 [23]) prunes tokens by perplexity or distillation metrics. It needs no architectural change but discards tokens irreversibly, breaking cross-token dependencies on relational tasks [13], with no on-demand dereferencing.

2.3 Continuous and Soft-Token Compression

Soft-token methods shorten the active sequence but provide no indirection. Gist Tokens [20] condense a prefix into virtual tokens that must stay *resident* in attention—unconditional compression with no per-query selection. AutoCompressors [3] recursively encode chunks into summary vectors that are *irrecoverable projections*: a later query cannot recover one sub-chunk without re-encoding. ICAE [6] uses fixed slots with implicit source binding, decoupled from query selection. 500xCompressor [15] collapses context into one token, so two facts cannot be recovered for two queries without re-encoding. None provides an explicit address level: unselected context stays resident or is discarded without a dereferencing handle.

2.4 Dynamic Cache Eviction and Paged KV Caches

Compressed Context Memory (CCM) [10] and Contextual Semantic Anchors (SAC) [17] evict or aggregate KV states; PagedAttention [12] pages KV blocks over non-contiguous memory. They ease GPU pressure, but eviction is irreversible: no token-level address dereferences an evicted state by content.

2.5 Hierarchical Context and OS-Style Memory

MemGPT [22] pages text chunks between active context and archival storage via function/tool calls, addressing pages by positional identifiers at the orchestration layer. SemPointer instead defines sequence-level indirection: content-derived token addresses ($P_i = \mathcal{G}_\phi(X_i)$) persist in active context while payloads sit in an independent substrate.

2.6 Cognitive Science Disambiguation: Semantic Pointer Architecture

The term *semantic pointer* originates in cognitive science: Eliasmith [4] and Blouw et al. [1] formalized the Semantic Pointer Architecture (SPA) with high-dimensional vectors ($D \approx 500\text{--}1000$) bound by circular convolution in spiking networks. SemPointer targets a disjoint domain—discrete or latent token sequences in autoregressive transformers—so the overlap is terminological only.

3 Axiomatic Foundations: Distinguishing Pointers from Compressed Representations

A compressed representation carries information inside active attention; a pointer is an address into external storage dereferenced explicitly. Definition 1 and seven axioms formalize the distinction.

Definition 1 (Compressed Representation vs. Semantic Pointer). *Let $X \in \mathcal{X}$ be an information unit of length $|X|$. A mapping $\mathcal{C} : \mathcal{X} \rightarrow \mathcal{Z}$ produces a compressed representation Z if $|Z| < |X|$ and Z directly acts as the information carrier in downstream computation. A mapping $\mathcal{G} : \mathcal{X} \rightarrow \mathcal{P}$ produces a semantic pointer P if P acts as an address into an independent memory substrate $\mathcal{M}$, such that the information content of X is accessed only upon executing an explicit resolution operation $\mathcal{R}(P, M, Q)$.*

Axiom 1 (Address Identity and Distinguishability) *Let $X_i, X_j \in \mathcal{X}$ be two semantically distinct context units ($X_i \not\equiv X_j$). Their generated pointers $P_i = \mathcal{G}(X_i)$ and $P_j = \mathcal{G}(X_j)$ must be distinguishable within the address space: $D_{\mathcal{P}}(P_i, P_j) > \delta_{\min}$ for all $i \neq j$, where $D_{\mathcal{P}}$ is a metric on pointer space $\mathcal{P}$ and $\delta_{\min} > 0$. Beyond injectivity, $\mathcal{G}_{\phi}$ must preserve semantic neighborhoods (similar blocks yield proximate pointers) to enable query-conditioned scoring; this alignment is a training objective, not a consequence of distinguishability. This axiom is not exercised by the present measurements (no pointer-distance statistics logged).*

Axiom 2 (Strict Asymmetric Compactness) *The pointer length $|P_i|$ measured in sequence units (tokens) must satisfy $|P_i| \ll |X_i|$ with $|P_i|/|X_i| \leq \kappa, \kappa \in (0, 1)$. The pointer representation must not scale with context-unit length. In the operating regime ($k \leq 8$, $|X_i| \geq 160$ tokens), $\kappa \leq 0.05$ holds by construction; units already at pointer scale gain nothing from indirection. Measured: $k = 8$ against true $B \approx 504\text{--}510$ gives $\kappa \approx 0.016$ (`config_table.json`; Section ??).*

Axiom 3 (Temporal and Spatial Stability) *A pointer P_i generated at time t must remain valid and addressable across subsequent time horizons $t + \Delta t$, independent of intervening context operations, memory updates, or unrelated token generations. Not exercised by the present measurements (single query per example; no position-displacement run).*

Axiom 4 (Query-Conditioned Resolution Fidelity) *Given query $Q \in \mathcal{Q}$, resolution operator $\mathcal{R}$ acting on pointer P_i and substrate $M_i = \mathcal{S}(X_i)$ must recover task-relevant mutual information:*

$$I(\mathcal{R}(P_i, M_i, Q); X_i \mid Q) \geq (1 - \epsilon)H(X_i \mid Q) \quad (2)$$

for tolerance $\epsilon \in [0, 1]$, where $H(\cdot)$ denotes conditional entropy. Lossless substrates ($\mathcal{M}_{\text{text}}$, exact $\mathcal{M}_{\text{kv}}$) give $\epsilon = 0$ by definition; lossy ones ($\mathcal{M}_{\text{latent}}$) admit $\epsilon \in (0, 1)$. The verbatim-text path is the measured one (Section ??; downstream deltas in Section 10.1); the ϵ -bound itself is not directly estimated.

Axiom 5 (Selective Addressability) *The activation of historical information must be sparse. Given a registry of N pointers $\{P_1, \dots, P_N\}$, a query Q_t resolves only the subset of indices $J^* \subseteq \{1, \dots, N\}$ where $|J^*| \ll N$, avoiding uniform activation of the remaining $N - |J^*|$ elements. Measured at $m = 1$ against $N \approx 24.8/26.7/28.0/7.5$ (`config_table.json`; miss attribution in Section 10.5).*

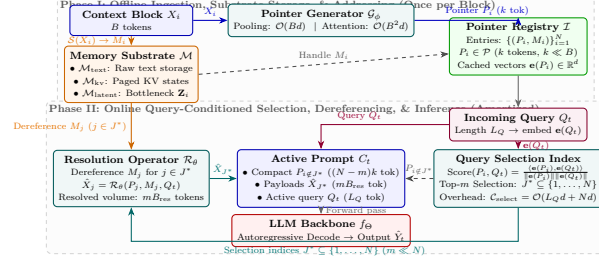


Fig. 1. The SemPointer architectural lifecycle and multi-tier memory substrate. (Top) Phase I (Offline Ingestion and Addressing): Context block X_i (B tokens) is committed to substrate $\mathcal{M}$ ($\mathcal{M}_{\text{text}}, \mathcal{M}_{\text{kv}}, \mathcal{M}_{\text{latent}}$) via $\mathcal{S}$, while generator $\mathcal{G}_\phi$ produces k -token pointer P_i , registered in index $\mathcal{I}$ with cached projection vector $\mathbf{e}(P_i) \in \mathbb{R}^d$. (Bottom) Phase II (Online Query Selection and Resolution): Query Q_t (L_Q tokens) scores against $\mathcal{I}$ to select sparse subset $J^* \subseteq \{1, \dots, N\}$ ($m \ll N$). Resolution operator $\mathcal{R}_\theta$ dereferences active blocks M_j into payloads $\hat{X}_j$, which combine with passive unselected pointers $P_{i \notin J^*}$ and query Q_t into active prompt C_t for autoregressive decode by LLM backbone f_θ .

Axiom 6 (Multi-Horizon Reusability) A pointer P_i must support repeated resolution operations across arbitrary non-consecutive queries $Q_{t_1}, Q_{t_2}, \dots, Q_{t_m}$ without requiring re-execution of pointer generation $\mathcal{G}(X_i)$. Not exercised by the present measurements (no repeated-resolution run).

Axiom 7 (Algebraic Compositionality) For compound queries requiring information from multiple context items $\{X_a, X_b\}$, the joint pointer representation must satisfy:

$$\mathcal{R}(P_a \oplus P_b, \{M_a, M_b\}, Q) \equiv \mathcal{R}(P_a, M_a, Q) \cup \mathcal{R}(P_b, M_b, Q) \quad (3)$$

where $\oplus : \mathcal{P} \times \mathcal{P} \rightarrow \mathcal{P}$ is an abstract composition operator. Axiom 7 requires composite dereferencing, specified as joint multi-pointer dereferencing ($[P_a, P_b] \mapsto [\hat{X}_a, \hat{X}_b]$) but not exercised by the present measurements (former Experiment E11 withdrawn, see Appendix; the multi-hop pilot favors dense top-1 over index routing, Table 8); an end-to-end $\oplus$ in latent space remains open.

4 The SemPointer Formal Architecture

4.1 Mathematical Model

The model operates over five spaces, depicted in Fig. 1 across its offline ingestion and online query lifecycles. Context Space $\mathcal{X}$ contains textual sequences $X \in \mathcal{V}^*$. Substrate Space $\mathcal{M}$ holds independent storage representations (raw text, KV tensors, or latent embeddings). Pointer Space $\mathcal{P}$ consists of sequences of k discrete tokens or soft embeddings, with k fixed and small ($k \in [1, 8]$). Query Space $\mathcal{Q}$ contains active prompts, and Output Space $\mathcal{Y}$ contains target generations.

1) Encounter and Substrate Ingestion: When context item X_i is encountered at time t_i , the storage operator $\mathcal{S}$ commits X_i to substrate $\mathcal{M}$ while pointer generator $\mathcal{G}_\phi$ produces the compact semantic pointer:

$$M_i = \mathcal{S}(X_i), \quad P_i = \mathcal{G}_\phi(X_i) = (p_{i,1}, \dots, p_{i,k}) \quad (4)$$

The pair (P_i, M_i) is registered in pointer index $\mathcal{I}_{t_i} = \mathcal{I}_{t_i-1} \cup \{(P_i, M_i)\}$.

2) Query-Conditioned Selection: Upon arrival of query Q_t , registered pointers are scored via normalized inner product:

$$s_i = \text{Score}(P_i, Q_t) = \frac{\langle \mathbf{e}(P_i), \mathbf{e}(Q_t) \rangle}{\|\mathbf{e}(P_i)\| \|\mathbf{e}(Q_t)\|} \quad (5)$$

where $\mathbf{e}(\cdot)$ denotes an addressing projection into a shared metric space. The system selects top- m pointers $J^* = \text{top-}m(\{s_i\}_{i=1}^N)$.

3) Pointer Dereferencing and Resolution: For each selected index $j \in J^*$, resolution operator $\mathcal{R}_\theta$ dereferences P_j against substrate M_j , conditioned on Q_t : $\hat{X}_j = \mathcal{R}_\theta(P_j, M_j, Q_t)$. Resolved information $\hat{X}_{J^*}$ is injected into the generation prompt alongside unselected pointers:

$$\hat{Y}_t \sim f_\theta \left(\cdot \mid [\{P_i\}_{i \notin J^*}, \hat{X}_{J^*}, Q_t] \right) \quad (6)$$

Because unselected blocks remain represented by their compact pointers P_i ($i \notin J^*$), active context consumes $L_{\text{active}} = (N - m)k + \sum_{j \in J^*} |\hat{X}_j| + |Q_t| \leq Nk + mB_{\text{res}} + L_Q$ tokens, compared to $NB + L_Q$ in linear re-ingestion. Because all $N - m$ unselected pointers remain in the sequence, attention across the active prompt remains quadratic in L_{active} ; SemPointer reduces sequence length by replacing unselected blocks of size B with pointers of size $k \ll B$, compressing the quadratic attention coefficient rather than altering the asymptotic polynomial degree.

4.2 Concrete Instantiation: Generator and Resolver Mechanics

Algorithm 1 formalizes the operational lifecycle implemented in our framework. To eliminate training artifacts and domain-specific bias, our primary realization employs a zero-shot, training-free pointer generator $\mathcal{G}_\phi$ and deterministic substrate resolvers $\mathcal{R}_\theta$:

- **Training-Free Generator ($\mathcal{G}_\phi$):** In the reported runs, dense addressing projection $\mathbf{e}(P_i)$ is generated via the frozen public encoder `sentence-transformers/all-MiniLM-L6-v2` with mean pooling over per-block window embeddings (Section ??); discrete prompt handles are seeded random-projection buckets over the same embedding (k tokens). Other generator realizations (attention-class readouts, TF-IDF handles, soft-prompt expansion) are specified as design alternatives in the cost model but were not executed. Because $\mathcal{G}_\phi$ operates entirely zero-shot over frozen pre-trained representations, parameter training requires zero optimization steps, zero fine-tuning data, and incurs zero unamortized training compute ($\mathcal{C}_{\text{train}} = 0$).

Algorithm 1 Lifecycle of SemPointer Addressing and Resolution.

Require: Stream $\mathcal{S}_{\text{stream}}$, Queries $\{Q_t\}$, Storage $\mathcal{S}$, Generator $\mathcal{G}_\phi$, Resolver $\mathcal{R}_\theta$, LLM f_Θ

- 1: Initialize Pointer Registry $\mathcal{I} \leftarrow \emptyset$
- 2: **for all** context blocks $X_i \in \mathcal{S}_{\text{stream}}$ **do**
- 3: $M_i \leftarrow \mathcal{S}(X_i)$; $P_i \leftarrow \mathcal{G}_\phi(X_i)$; $\mathcal{I} \leftarrow \mathcal{I} \cup \{(P_i, M_i)\}$
- 4: **end for**
- 5: **for all** queries Q_t **do**
- 6: Score $s_i = \text{Score}(P_i, Q_t)$ for $(P_i, M_i) \in \mathcal{I}$; $J^* \leftarrow \text{top-}m(\{s_i\}_{i=1}^N)$
- 7: Active memory payload $\Omega_t \leftarrow \bigcup_{j \in J^*} \{\mathcal{R}_\theta(P_j, M_j, Q_t)\}$
- 8: Active prompt $C_t \leftarrow [\{P_i\}_{i \notin J^*}, \Omega_t, Q_t]$; Emit $\hat{Y}_t \leftarrow f_\Theta(C_t)$
- 9: **end for**

- **Deterministic Resolvers ($\mathcal{R}_\theta$):** For text substrate $\mathcal{M}_{\text{text}}$, $\mathcal{R}_\theta$ performs exact in-memory index dereferencing returning the verbatim string X_i . For key-value cache $\mathcal{M}_{\text{kv}}$, $\mathcal{R}_\theta$ performs physical memory block dereferencing, injecting cached KV tensors into attention tables ($\mathcal{C}_{\text{res}} \approx 0$). Neither resolver requires learned parameters, guaranteeing exact resolution fidelity ($\epsilon = 0$).

5 Memory Substrates and Physical Taxonomies

Three physical realizations of M_i trade off cost, capacity, and fidelity.

A. Raw Text Substrate ($\mathcal{M}_{\text{text}}$): Holds $M_i = X_i$ in flat storage (RAM/disk); activation fetches X_i into context. No encoding penalty or fidelity loss, but resolution pays I/O plus re-encoding.

B. Key-Value Cache Substrate ($\mathcal{M}_{\text{kv}}$): Here $M_i = (\mathbf{K}_i, \mathbf{V}_i) \in \mathbb{R}^{2 \times L_{\text{layers}} \times |X_i| \times d}$, precomputed in host or paged GPU memory [12]; resolution pages KV blocks in with $\mathcal{C}_{\text{res}} \approx 0$ FLOPs. Capacity is the constraint: at fp16, a $B=512$ block in a representative 70B-scale configuration ($n_{\text{layers}}=80, n_{\text{heads}}=64, d_{\text{head}}=128$) occupies ≈ 1.34 GB by direct KV-tensor arithmetic ($2 \text{ bytes} \times 2 \text{ tensors} \times \text{layers} \times \text{tokens} \times \text{heads} \times \text{head-dim}$), so $N \geq 10^3$ blocks need terabyte-scale storage. RoPE displacement Δt can further degrade relocated blocks.

C. Latent Embedding Substrate ($\mathcal{M}_{\text{latent}}$, future work): $M_i = \mathbf{Z}_i \in \mathbb{R}^{m \times d}$ with $m \ll |X_i|$ would come from an autoencoder [6], with resolution by cross-attention unpooling over $\mathbf{Z}_i$ conditioned on P_i, Q_t ; compact but lossy. Future work only: no such module is trained and no reported run exercises it (all measurements use $\mathcal{M}_{\text{text}}$, Section ??).

6 Theoretical Complexity and Cost Modeling

6.1 Modeling Assumptions

We analyze indirection against linear re-ingestion under five assumptions:

- A1. Uniform Block Partitioning:** Context splits into N blocks of length B ($L_H = N \cdot B$).
- A2. Sparse Access Frequency:** Across K queries $\{Q_1, \dots, Q_K\}$ (each length L_Q), each query accesses on average $m \ll N$ units (deterministic expectation; per-query variance unmodeled).

Property	$\mathcal{M}_{\text{text}}$ (Raw Text)	$\mathcal{M}_{\text{kv}}$ (KV Cache)	$\mathcal{M}_{\text{latent}}$ (Latent)
Storage	Flat text buffer (RAM/disk)	$(\mathbf{K}_i, \mathbf{V}_i) \in \mathbb{R}^{2 \times L \times X_i \times d}$, pre-computed	$\mathbf{Z}_i \in \mathbb{R}^{m_z \times d}$, $m_z \ll X_i $
Resolution	Re-tokenize & re-ingest X_i	Page KV blocks into attention tables	Cross-attention unpooling over $\mathbf{Z}_i$
$\mathcal{C}_{\text{res}}$	$\mathcal{O}(B_{\text{res}} d^2)$ (re-encode)	≈ 0 FLOPs	$\mathcal{O}(k m_z d)$
Fidelity	Lossless ($\varepsilon \approx 0$)	Lossless; RoPE displacement risk	Lossy ($\varepsilon > 0$, bottleneck-dependent)
Bottleneck	I/O latency & encoding cost	Memory capacity & paging bandwidth	Reconstruction fidelity; hallucination risk

Table 2. The three SemPointer memory substrates compared. All decouple pointer generation from payload storage; they differ in resolution cost, storage volume, and fidelity. $\mathcal{M}_{\text{kv}}$ eliminates recompute FLOPs but is capacity-bound; $\mathcal{M}_{\text{latent}}$ minimizes storage at the cost of reconstruction loss; $\mathcal{M}_{\text{text}}$ is lossless but re-incurs encoding cost on each resolution.

- A3. Compact Pointer Invariant:** Each pointer is k tokens with $k \ll B$ (Axiom 2).
A4. Upfront Amortization: Initial block encoding and pointer generation ($\mathcal{C}_{\text{init}}$) run once at encounter and amortize over later queries.
A5. Cached Addressing Embeddings: Projections $\mathbf{e}(P_i)$ are cached at registration; scoring needs one $\mathbf{e}(Q_t)$ plus N inner products, so $\mathcal{C}_{\text{select}} = \mathcal{O}(L_Q d + N d)$ per (5), as required by Axioms 3–6. Fine-tuning the encoder invalidates the cache.

6.2 Resolution Cost by Substrate

Resolution cost $\mathcal{C}_{\text{res}}(m, B_{\text{res}}, Q_t)$ follows Section 5: $\mathcal{M}_{\text{text}}$ re-encodes fetched text, $\mathcal{C}_{\text{res}} = m(2dB_{\text{res}}^2 + 4d^2B_{\text{res}})$; $\mathcal{M}_{\text{kv}}$ reuses paged KV states ($\mathcal{C}_{\text{res}} \approx 0$ FLOPs; bandwidth- and capacity-bound); $\mathcal{M}_{\text{latent}}$ cross-attends over k pointer tokens, $\mathcal{C}_{\text{res}} = \mathcal{O}(mB_{\text{res}}kd)$.

6.3 Asymptotic FLOPs Analysis

Consider an LLM with hidden dimension d and multi-head self-attention.¹ We compare cost across K queries under re-ingestion versus SemPointer.

- 1) **Linear Re-Ingestion Paradigm:** Each query re-attends to the full history L_H :

$$\begin{aligned} \mathcal{C}_{\text{linear}} &= K [2d(NB + L_Q)^2 + 4d^2(NB + L_Q)] \\ &\approx 2Kd(NB + L_Q)^2 \end{aligned} \quad (7)$$

scaling as $\mathcal{O}(KdN^2B^2)$ to leading order.

- 2) **SemPointer Indirection Paradigm:** Under SemPointer:

1. **One-Time Generation and Storage:** Ingesting N blocks and generating k -token pointers incurs:

$$\mathcal{C}_{\text{init}} = N \cdot [2dB^2 + 4d^2B + \mathcal{C}_{\text{gen}}(B, k)] \quad (8)$$

¹ We suppress layer-dependent constant factors ($n_{\text{layers}}, n_{\text{heads}}$); see Section 2.1 for the expanded KV-state expression.

Note $4d^2B$ dominates $2dB^2$ whenever $B < 2d$ (typical: $B \leq 10^3$, $d \geq 4096$). Generator cost $\mathcal{C}_{\text{gen}}$ depends on $\mathcal{G}_\phi$: pooling-class readouts cost $\mathcal{O}(Bd)$; attention readouts (Corollary 1) cost $\mathcal{C}_{\text{gen}} = c_g(2dB^2 + 4d^2B)$ with $c_g = \Theta(1)$, roughly doubling $\mathcal{C}_{\text{init}}$ at $c_g \approx 1$. Our pooling generator is training-free ($\mathcal{C}_{\text{train}} = 0$), and $\mathcal{C}_{\text{init}}$ is one-time ingestion plus addressing. Proposition 2 covers the pooling-class lower bound ($\mathcal{C}_{\text{gen}} \geq \Omega(Bd)$); Corollary 1 covers attention-class generators.

2. **Per-Query Pointer Selection:** $\mathcal{C}_{\text{select}} = \mathcal{O}(L_Q d + Nd)$ (Eq. (5), A5).
3. **Per-Query Resolution and Inference:** The active prompt holds $N - m$ pointers of length k , m resolved blocks ($B_{\text{res}} \leq B$), and Q_t :

$$L_{\text{active}} = (N - m)k + mB_{\text{res}} + L_Q \leq Nk + mB_{\text{res}} + L_Q \quad (9)$$

Per-query computation is:

$$\mathcal{C}_{\text{query}} = 2d L_{\text{active}}^2 + 4d^2 L_{\text{active}} + \mathcal{C}_{\text{res}}(m, B_{\text{res}}, Q) \quad (10)$$

Summing over K queries:

$$\begin{aligned} \mathcal{C}_{\text{SemPointer}} &= \mathcal{C}_{\text{init}} + K[\mathcal{C}_{\text{select}} + \mathcal{C}_{\text{query}}] \\ &= N(2dB^2 + 4d^2B + \mathcal{C}_{\text{gen}}) \\ &\quad + K[2dL_{\text{active}}^2 + 4d^2L_{\text{active}} + \mathcal{C}_{\text{res}} + \mathcal{C}_{\text{select}}] \end{aligned} \quad (11)$$

6.4 The Indirection Crossover Threshold

Proposition 1 (Crossover Threshold). *Under A1–A5, indirection yields lower cumulative FLOPs than linear re-ingestion iff (i) $\Delta\mathcal{C}_{\text{step}} > 0$ — up to overheads, equivalently $N(B - k) > mB_{\text{res}}$ — and (ii) $K > K^* = \mathcal{C}_{\text{init}}/\Delta\mathcal{C}_{\text{step}}$:*

$$K^* = \frac{\mathcal{C}_{\text{init}}}{\Delta\mathcal{C}_{\text{step}}} \quad (12)$$

where the per-query marginal savings $\Delta\mathcal{C}_{\text{step}}$ is given by:

$$\begin{aligned} \Delta\mathcal{C}_{\text{step}} &= [N(B - k) - m(B_{\text{res}} - k)] \\ &\quad \cdot [2d(N(B + k) + m(B_{\text{res}} - k) + 2L_Q) + 4d^2] \\ &\quad - \mathcal{C}_{\text{select}} - \mathcal{C}_{\text{res}} \end{aligned} \quad (13)$$

Condition (i) makes the sparsity requirement of A2 concrete: if a query resolves a large fraction of memory ($m(B_{\text{res}} - k) \geq N(B - k)$), indirection is strictly dominated for every K . Both K^* and $\Delta\mathcal{C}_{\text{step}}$ are analytic derivations: no K -sweep or FLOPs-crossover fit is measured (measured operating points are overlaid on the analytic map in Section 10.8).

Derivation. With $L_{\text{active}} = Nk + m(B_{\text{res}} - k) + L_Q$, difference-of-squares gives $(L_H + L_Q)^2 - L_{\text{active}}^2 = [N(B - k) - m(B_{\text{res}} - k)][N(B + k) + m(B_{\text{res}} - k) + 2L_Q]$; adding linear projection difference $4d^2[N(B - k) - m(B_{\text{res}} - k)]$ yields (13), i.e. (tokens avoided) $\times$ (per-token coefficient) minus addressing and resolution overheads. Per-call end-to-end latency is logged (`GenResult.latency_ms`, with a TTFT tap in `experiments/engine/llm.py`) and reported as means in Table 6, without a FLOPs-crossover fit.

Table 3. Architectural Comparison of Context-Management and Memory Approaches. Entries characterize each system at the level of its published description; see Section VIII-B for a comparison of composable addressing (Axiom 7) against standard RAG concatenation.

Framework	Primary Representation	Representation Type	Decoupled Substrate?	Explicit Address?	Selective Access?	Composable Addressing?	Multi-Turn Reusability?
Gist Tokens [20]	Soft Tokens	Continuous	No	No	No	No	Yes
AutoCompressors [3]	Summary Vectors	Continuous	No	No	No	No	Yes
ICAE [6]	Memory Slots	Continuous	No	No	No	No	Yes
500xCompressor [15]	Special Tokens	Discrete/Latent	No	No	No	No	No
SAC [17]	Anchor Tokens	Token KV	No	No	No	No	No
CCM [10]	Compressed KV	KV Tensors	Yes (KV)	No	No	No	Yes
LLMLingua-2 [23]	Pruned Text	Discrete Text	No	No	No	No	No
xRAG [2]	Document Embedding	Continuous	Yes (Text DB)	Partial	Yes (Retrieval)	No	Yes
MemGPT [22]	Page ID / Chunks	Positional/Text	Yes (Archival)	Partial (Positional)	Yes (Tool-Call)	No	Yes
Landmark Attention [19]	Landmark Tokens	Token Markers	No	No	Yes (Blocks)	No	No
SemPointer (Proposed)	Semantic Pointer	Token Seq. / Latent	Yes (Multi-tier)	Yes (Semantic)	Yes (Query-Cond.)	Yes (Design req.)	Yes

Proposition 2 (Single-Query Viability). *Under A1–A5 with the $\mathcal{M}_{kv}$ substrate ($\mathcal{C}_{res} \approx 0$), $L_Q \ll NB$, and sparse access ($mB_{res} \ll NB$), the condition $K^* < 1$ (single-query viability) reduces to $N > 1 + \mathcal{O}(k/B + dk/B^2)$. Under pooling-class generation, this condition is satisfied for integer registry sizes $N \geq 2$. At $N = 1$, indirection is marginal and does not amortize on the first query ($K^* \approx 1.015 > 1$ for $d = 4096$, $B = 512$, $k = 8$, $\mathcal{C}_{gen} = Bd$), whereas crossover to single-query viability occurs at $N = 2$ ($K^* \approx 0.958 < 1$). Corollary 1 treats attention-class generators. Sparsity of access ($m \ll N$), rather than query reuse count, is the load-bearing condition under $\mathcal{M}_{kv}$. This threshold illustration is analytic (no $N=1$ -vs-2 run exists). Under resolution-heavy substrates ($\mathcal{M}_{text}$), where $\mathcal{C}_{res}$ reinstates quadratic cost per query, K^* exceeds 1 and multi-query amortization governs.*

Corollary 1 (Attention-Class Pointer Generators). Proposition 2 assumes a pooling-class generator with $\mathcal{C}_{gen} = \mathcal{O}(Bd)$ (e.g., readout pooling over precomputed token embeddings). If $\mathcal{G}_\phi$ is an attention-based readout, $\mathcal{C}_{gen} = c_g(2dB^2 + 4d^2B)$, $c_g = \Theta(1)$ — the same asymptotic order as block encoding — and the viability condition reduces, to leading order, to $N > (1 + c_g) + (2d/B)c_g$. For $c_g = 1$: $N \gtrsim 2 + 2d/B$ (e.g., $N \gtrsim 18$ at $d=4096$, $B=512$; $N \gtrsim 6$ at $B=2048$). This is a meaningful degradation from the pooling-class threshold of $N \gtrsim 2$: an attention-based generator operating at $B=512$ demands a registry roughly nine times larger before single-query viability holds. The threshold nonetheless grows only linearly with d/B and remains finite, so viability is preserved for practical N ; below the threshold, the framework falls back to the multi-query amortization regime of Proposition 1. Analytic only: no attention-class generator is built or timed in the present runs.

Remark 1 (Scope of Comparison). $\mathcal{C}_{linear}$ models the no-cache regime: stateless inference, API access, cache eviction between sessions, or contexts exceeding cache capacity. Against a persistent KV cache with host offload [12], per-query recompute FLOPs approach zero and the binding resource becomes memory capacity and paging bandwidth. In that regime SemPointer’s contribution is capacity, not FLOPs: retaining only Nk pointer tokens in fast memory while full context resides in secondary storage. The crossover analysis covers the re-ingestion regime; the capacity argument extends to the cached regime.

7 Theoretical Vulnerabilities and Failure Modes

A. Address Collision via Generalized Birthday Bounds:

Proposition 3 (Combinatorial Address Collision Bound). *Let pointer space $\mathcal{P}$ consist of sequences of length k over discrete vocabulary $\mathcal{V}_p$ of size $V = |\mathcal{V}_p|$. Under*

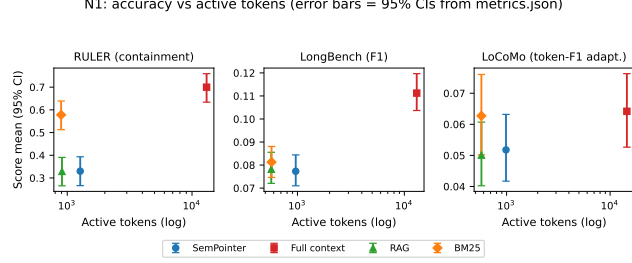


Fig. 2. Lead result: measured accuracy versus measured active tokens (Pareto plane), one panel per suite, with 95% CIs. SemPointer sits an order of magnitude left of full context on tokens at lower accuracy, and ties dense RAG at higher token cost. KV-compress pilot omitted (n=10).

independent, uniformly distributed pointer generation, the probability that any two of N registered memory items share an identical address is bounded by $P_{\text{collision}} \approx 1 - \exp\left(-\frac{N(N-1)}{2V^k}\right)$.

Proof Sketch. Generalized birthday bound over address cardinality $|\mathcal{P}| = V^k$ under independent uniform assignment. The uniform assumption is contradicted by the implementation note that equal-width binning yields a non-uniform bucket distribution (`pointer_generator.py`); no collision rate is logged.

Proposition 4 (Combinatorial Address Space Capacity). *Under the uniform birthday-bound model, to maintain collision probability below threshold δ across a memory registry of N items, the minimum discrete pointer length k must satisfy $k \geq \left\lceil \frac{\ln(N^2) - \ln(2\delta)}{\ln V} \right\rceil$. For illustration, with $N = 10^5$, $V = 32,000$, and $\delta = 10^{-4}$: $k \geq \left\lceil \frac{23.02 + 8.51}{10.37} \right\rceil = 4$. This calculation concerns combinatorial capacity; Proposition 4’s lower bound ($k \geq 4$ under $N = 10^5$, $V = 32,000$, $\delta = 10^{-4}$) and Axiom 2’s compactness jointly restrict viable pointer length to $k \in [4, 8]$. Analytic illustration only: all reported runs use $k = 8$, which satisfies the bound without testing it (`config_table.json`).*

B. Semantic Drift Across Extended Horizons (assumption; future measurement): We assume, as a phenomenological ansatz rather than a derivation from RoPE geometry, that attention to historical pointers may decay with displacement $\Delta t = t_{\text{curr}} - t_{\text{gen}}$ as: $\mathbb{E}[\alpha(Q_t, P_i)] \propto \exp(-\lambda|t - t_i|)$, where $\lambda > 0$ is an unspecified decay parameter. No position-displacement measurement exists in the present runs; testing this decay is future work.

C. Resolution Divergence and Hallucination: When $\mathcal{R}_\theta$ operates on a lossy substrate ($\mathcal{M}_{\text{latent}}$), reconstructions may deviate from ground truth: $\mathcal{D}_{\text{KL}}(p(X_i) \parallel p(\hat{X}_i | P_i, M_i)) > 0$. Beyond downstream reasoning tolerance this hallucinates, so precision-critical tasks assume $\mathcal{M}_{\text{text}}$ or $\mathcal{M}_{\text{kv}}$; no $\mathcal{M}_{\text{latent}}$ run exists to test this (future work, Section 5).

8 Critical Boundary Analysis and Literature Comparison

A. Disambiguation from Landmark Attention: Landmark Attention [19] uses tokens to route attention across blocks. Three structural differences separate it from

SemPointer: (1) *Sequence routing vs. persistent indirection*: Landmark Attention gates attention within a continuous sequence; SemPointer maintains an address registry dereferencing an independent substrate across queries. (2) *Internal marker vs. address handle*: Landmark tokens act as transient routing gates; SemPointers are persistent, composable address handles. (3) *Working-memory eviction*: Landmark Attention requires keeping full context or KV states active; SemPointer permits offloading substrate to secondary storage while retaining only k pointer tokens per block.

B. Disambiguation from Retrieval-Augmented Generation: While RAG [2,14] and SemPointer both perform query-conditioned selection over external stores, they diverge fundamentally in addressing structure. Standard RAG retrieves passages as independent blocks and appends them to prompt context, delegating relational binding entirely to downstream self-attention. SemPointer defines an address space of composable references ($P_a \oplus P_b$), enabling compound pointers to be registered and dereferenced jointly (Axiom 7, (3)). Joint resolution is specified but not empirically tested in this work: the multi-hop pilot (Table 8) favors dense top-1 over index-routed resolution. For a two-hop query (“*What did the vendor in turn 12 quote in turn 40’s email?*”), RAG concatenates both blocks into context, whereas SemPointer forms a joint address ($P_{12} \oplus P_{40}$) resolved once against combined substrate $\{M_{12}, M_{40}\}$ conditioned on Q_t .

C. Disambiguation from Operating-System-Style Memory (MemGPT): MemGPT [22] manages context hierarchies through operating-system abstractions, moving text blocks between active context and archival storage via function calls and parser interrupts. Three structural distinctions separate SemPointer from MemGPT: (1) *Addressing abstraction*: MemGPT addresses storage items via integer page IDs or text metadata mediated by system prompt instructions and external tool calls. SemPointer introduces token-level addresses generated directly by $\mathcal{G}_\phi$ within the model’s sequence space. (2) *Active sequence integration*: In MemGPT, evicted pages leave no residual trace in active context; retrieval requires explicit tool invocation. In SemPointer, all unselected blocks retain compact k -token pointers ($P_{i \notin J^*}$) in active attention, preserving global context presence while payloads remain offloaded. (3) *Formal complexity boundary*: While MemGPT is an empirical systems framework, SemPointer provides closed-form cost derivations (Propositions 1–2) determining when address indirection amortizes against linear re-ingestion.

D. Feature-Level Comparison with Related Approaches: Table 3 contrasts SemPointer with representative context-compression and memory architectures across seven structural dimensions. **Explicit Address and Decoupled Substrate:** Gist Tokens [20, §2], AutoCompressors [3, §3.1], and ICAE [6, §2] are address-less prompt prefixes with no substrate. 500xCompressor [15, §3] collapses context into one token. CCM [10, §3.2] offloads merged KV states (*Yes (KV)*) with no explicit address (*No*). xRAG [2, §2.2] retrieves by dense vector (*Partial address*, *Yes (Ext DB)* substrate). MemGPT [22] uses page-level access (*Partial address*, *Yes (Archival)* substrate). **Composable Addressing (Axiom 7):** AutoCompressor vectors [3, §3.1] lack an address algebra for non-adjacent references (*No*). Gist Tokens [20] and ICAE [6] leave binding to attention over summaries. Landmark Attention [19, §3] routes within-sequence with no cross-turn handles (*No*). SemPointer instead requires composition ($P_a \oplus P_b$, (3)).

E. Query-Aware Sparsity and KV-Reuse Systems: Three recent systems implement the dense-selection half of SemPointer’s pipeline and bound its novelty claims. InFLM [34] pairs sliding-window attention with block-level KV memory whose representatives are selected by attention scores, scaling to 1M tokens with no persistent in-prompt handle. RetrievalAttention [16] offloads KV to CPU behind ANN indexes with per-step retrieval scanning 1–3% of keys; its addresses are transient index hits, not authored tokens. Quest [27] loads Top- K KV pages per query via analytic attention bounds, likewise leaving no residual handle. CacheBlend [37] reuses precomputed KV of

non-prefix chunks with selective recompute repairing cross-attention, keyed by chunk hash without query-conditioned selection. PromptCache [7] stitches position-correct cached KV modules across requests. Memorizing Transformers [31] bury kNN memory reads inside one attention layer with no textual address. Infini-attention [21] folds past key-value states incrementally into a fixed-size compressive associative matrix inside each attention block, retrieved by linear-attention lookup with a learned gate mixing long-term and local outputs. A second family pairs retrieval with compressed or embedded context and amortized or offline encoding across queries [5, 24–26, 35], while token-level reuse across requests targets decode acceleration [18]. Against all six, SemPointer’s defensible residual is narrow: prompt-resident textual pointer strings, verbatim-text (not KV) resolution, and a specified-but-untested composition algebra.

9 Empirical Methods

9.1 System under test

Context is chunked into blocks of at most $B = 512$ tokens (reader tokenizer; a trailing fragment shorter than $B/8$ merges into the previous block). Each block carries one $k = 8$ pointer; all final runs resolve $m = 1$ block per query (RAG baselines likewise retrieve $m = 1$). Resolution uses $\mathcal{M}_{\text{text}}$: the selected block’s verbatim text enters the prompt. Selection is dense cosine similarity between mean-pooled query and per-block pointer embeddings from the frozen public `sentence-transformers/all-MiniLM-L6-v2` encoder; blocks past the encoder’s maximum length are embedded whole via windowed mean-pooling (`embed_long`), never truncated. Pointer identifier strings (`[PTR_id: ...]`) are prompt-resident for unselected blocks only, carry no key–value state, and do not affect selection; dense RAG sees identical representations. KV ($\mathcal{M}_{\text{kv}}$) and latent ($\mathcal{M}_{\text{latent}}$) substrates are future work only: the KV-compress pilot (Table 7) runs KVPress methods as black-box baselines at $n = 10$, not a SemPointer KV substrate. Four further implemented baselines (agentic pointer routing, iterative RAG, hybrid fusion, session summaries) were not in the final runs and carry no numbers here.

9.2 Reader, suites, and metric adaptations

All suites run on one reader (`Qwen/Qwen2.5-3B-Instruct`, NF4, greedy decoding, seed 42) on a single 8 GB GPU with CPU-only embedding fallback. RULER ($n = 180$ per system) re-implements the official generators and applies the official per-sample containment rule (1.0 if the reference string appears, else 0.0). LongBench ($n = 1150$; 1136 scored for full context after 14 narrativeqa OOMs, see §10.6) spans six tasks (hotpotqa [36], 2wikimqa, musique [28], multifieldqa_en, narrativeqa [11], qasper) on official data, scored by verbatim official normalization plus Counter-based token F1 maximized over ground truths. LoCoMo ($n = 199$ per system, ten official conversations) uses a stated token-F1 adaptation for *all* categories—the official GPT-4 judge plus Porter-stemmed F1 is unavailable—so no LoCoMo number here is comparable to published LoCoMo results.

9.3 Statistics and configuration ledger

Comparisons use example-identifier-matched rows only (exact McNemar for binary containment, Wilcoxon signed-rank for F1; paired bootstrap 95% CIs with 10 000 resamples at seed 42; Bonferroni $\alpha = 0.05/13 \approx 0.0038$). True mean tokens per block are

506.6/504.2/510.2/487.9 (RULER/LongBench/LoCoMo/KV pilot), so $\kappa = k/B_{\text{true}} \approx 0.016$ at $k = 8$; mean blocks per example are 24.75/26.67/28.0/7.5. Recomputed score means match `metrics.json` on every suite. Full protocols, code paths, and deviations are in the supplement.

10 Results

10.1 Suite summary: accuracy losses first

Table 4 gives suite means with bootstrap 95% CIs; paired deltas, tests, and the Bonferroni family (13 tests, adjusted $\alpha \approx 0.0038$) come from `stats.json`. Three of the 13 paired tests survive correction: RULER SemPointer-vs-full, RULER SemPointer-vs-BM25, and LongBench SemPointer-vs-full [ESTABLISHED].

On RULER containment ($n = 180$ matched pairs), SemPointer scores 0.3300 against 0.7000 for full context: paired delta -0.37 , 95% CI $[-0.44, -0.30]$, McNemar discordant $b = 1$, $c = 61$, $p \approx 6.7 \times 10^{-14}$, surviving Bonferroni [ESTABLISHED]. Against BM25 (0.5778) the delta is -0.248 , CI $[-0.314, -0.183]$, $p \approx 2.3 \times 10^{-10}$, likewise surviving [ESTABLISHED]. Against dense RAG (0.3289), the pointer layer contributes essentially nothing: the delta is $+0.0011$, CI $[-0.003, +0.006]$, with zero McNemar discordant pairs ($b = c = 0$, $p = 1.0$; Wilcoxon $p = 0.65$) [ESTABLISHED]. The RAG and SemPointer rows are near-identical across the whole taxonomy (Table 9: both hit 57, miss 123, retrieval-miss 51) [ESTABLISHED].

On LongBench F1, SemPointer (0.0773, $n = 1150$) trails full context (0.1112, $n = 1136$ scored) by a paired delta of -0.033 , CI $[-0.040, -0.027]$ over $n = 1136$ matched pairs, Wilcoxon $p \approx 1.6 \times 10^{-31}$, surviving Bonferroni [ESTABLISHED]. SemPointer-vs-RAG is -0.0009 , CI $[-0.0047, +0.0028]$, $p = 0.32$, and SemPointer-vs-BM25 is -0.0040 , CI $[-0.010, +0.002]$, $p = 0.156$: neither is significant [ESTABLISHED]. Figure 2 places these accuracy–cost trade-offs on one plane.

On LoCoMo with the token-F1 adaptation ($n = 199$ matched pairs), SemPointer (0.0518) trails full context (0.0642) by -0.012 , CI $[-0.022, -0.003]$, Wilcoxon $p = 0.028$ — nominally positive but *not* significant after Bonferroni, so no difference is claimed [ESTABLISHED]. SemPointer-vs-RAG ($+0.0017$, CI $[-0.005, +0.008]$, $p = 0.69$) and SemPointer-vs-BM25 (-0.011 , CI $[-0.022, +0.001]$, $p = 0.037$) are likewise not significant after correction [ESTABLISHED]. No superiority over BM25 or full context is claimed on any suite [REASONED].

10.2 RULER per-task: easy parity, hard collapse

Table 5 and Figure 3 break RULER into per-task cells of $n = 30$. On the easiest needle task, `niah_single_1`, SemPointer matches full context exactly ($0.9667 = 0.9667$) [ESTABLISHED]. Difficulty then separates the systems: `niah_single_2` gives 0.2000 against full context’s 1.0000, `niah_single_3` gives 0.2667 against 0.7000, and `multikey_1` gives 0.4667 against 0.9333 [ESTABLISHED]. Variable tracking (`vt`) and common-word extraction (`cwe`) are collapses for every sparse system: SemPointer scores 0.0733 on `vt` against 0.5933 for full context, while `cwe` sits at the floor (0.0067) for all four systems including full context, a task-level bound rather than a retrieval effect [ESTABLISHED]. BM25 exceeds SemPointer on the harder NIAH cells (0.8000 and 0.6667 on `single_2/single_3`), consistent with lexical needles that survive sparse selection poorly under dense cosine at $m = 1$ [REASONED].

Table 4. Main results. Scores are suite means with 95% bootstrap CIs from `metrics.json`; paired deltas/p-values are in `stats.json`. RULER: RULER containment, n=180 (full n=180); LongBench: LongBench F1, n=1150 (full n=1136); LoCoMo: LoCoMo (token-F1 adaptation), n=199 (full n=199); LongBench full-context scored n=1136 vs 1150 elsewhere: 14 narrativeqa full-context OOMs excluded from scored means (OOM asymmetry: scored-only full mean 0.1112 vs errors-as-zero 0.1099; scored-only drops the hardest full-context cases, favoring full). KV-compress comparison is a separate pilot (Table N4, n=10) and not pooled here.

Suite	Metric	System	<i>n</i>	Score mean	95% CI	Active tok.	Latency (ms) / VRAM (MB)
RULER	RULER containment	SemPointer	180	0.3300	[0.2661, 0.3934]	1270.3	894.5 / 2331.2
		Full context	180	0.7000	[0.6339, 0.7595]	13089.2	7256.9 / 3463.0
		RAG	180	0.3289	[0.2655, 0.3906]	907.2	793.7 / 2295.2
		BM25	180	0.5778	[0.5128, 0.6389]	893.1	765.6 / 2294.0
LongBench	LongBench F1	SemPointer	1150	0.0773	[0.0710, 0.0844]	963.5	1527.2 / 2300.5
		Full context	1136	0.1112	[0.1037, 0.1197]	13066.1	8084.5 / 2994.4
		RAG	1150	0.0782	[0.0721, 0.0855]	568.8	1228.8 / 2263.6
		BM25	1150	0.0813	[0.0746, 0.0881]	572.7	1228.1 / 2263.9
LoCoMo	LoCoMo (token-F1 adaptation)	SemPointer	199	0.0518	[0.0417, 0.0632]	987.3	1308.8 / 2302.6
		Full context	199	0.0642	[0.0526, 0.0763]	14525.1	5326.2 / 3825.1
		RAG	199	0.0501	[0.0402, 0.0607]	571.5	1426.1 / 2263.8
		BM25	199	0.0627	[0.0502, 0.0760]	571.1	1425.7 / 2263.8

Table 5. RULER per-task containment means ($n = 30$ cells). Columns are tasks `niah_single_1/2/3`, `multikey_1`, `vt` (variable tracking), `cwe` (common-word extraction). Per-task bootstrap CIs are not stored in `metrics.json` (suite-level CIs only); Fig. N2 shows Wilson-95 whiskers computed in-script from cell means. This table exposes the `vt/cwe` weakness: all sparse systems collapse on `vt` and `cwe` while full context keeps a large `vt` margin.

System	niah_single_1	niah_single_2	niah_single_3	multikey_1	vt	cwe
SemPointer	0.9667 (30)	0.2000 (30)	0.2667 (30)	0.4667 (30)	0.0733 (30)	0.0067 (30)
Full context	0.9667 (30)	1.0000 (30)	0.7000 (30)	0.9333 (30)	0.5933 (30)	0.0067 (30)
RAG	0.9667 (30)	0.2000 (30)	0.2667 (30)	0.4667 (30)	0.0667 (30)	0.0067 (30)
BM25	0.9667 (30)	0.8000 (30)	0.6667 (30)	0.9333 (30)	0.0933 (30)	0.0067 (30)

10.3 Efficiency: large token and latency reductions at lower accuracy

Table 6 and Figure 4 report measured active tokens, end-to-end latency, and peak VRAM with sparse/full ratios. SemPointer uses roughly one-tenth the active context of full ingestion: 1270.3 vs 13089.2 tokens on RULER (ratio 0.097), 963.5 vs 13066.1 on LongBench (0.074), and 987.3 vs 14525.1 on LoCoMo (0.068) — about 10–15 $\times$ fewer tokens [ESTABLISHED]. Latency falls with prompt length: 894.5 vs 7256.9 ms on RULER (0.123, about 8 $\times$), 1527.2 vs 8084.5 ms on LongBench (0.189, about 5 $\times$), and 1308.8 vs 5326.2 ms on LoCoMo (0.246, about 4 $\times$) — 5–8 $\times$ faster end to end on RULER and LongBench [ESTABLISHED]. Peak VRAM drops from 3463.0 to 2331.2 MB on RULER (0.673), 2994.4 to 2300.5 MB on LongBench (0.768), and 3825.1 to 2302.6 MB on LoCoMo (0.602) [ESTABLISHED]. These savings accompany the accuracy losses of §10.1; they do not offset them, and the paper makes no net-benefit claim [REASONED]. The pointer layer itself is a small net cost over plain RAG: SemPointer feeds +363 more active tokens per request than RAG on RULER (1270.3 vs 907.2), about +395 on LongBench and +416 on LoCoMo, for a measured RULER delta of +0.001 [ESTABLISHED].

10.4 KV-compress pilot: no claims at $n = 10$

Table 7 reports the kv-compress pilot at $n = 10$ with wide CIs shown as-is. SemPointer scores 0.8000 (CI [0.60, 1.00]) against 1.0000 for full context (delta -0.2 , CI $[-0.5, 0.0]$, McNemar $p = 0.48$); against KVPress SnapKV 0.3 (0.5000) the delta is $+0.3$, CI $[-0.1, +0.7]$, $p = 0.37$; against ToVA 0.3 (0.8000) it is 0.0, CI $[-0.3, +0.3]$; against KNorm 0.3 (0.4000) it is $+0.4$, CI $[+0.1, +0.7]$, McNemar $p = 0.13$ [ESTABLISHED]. None of the four pilot tests survives Bonferroni, and the pilot note in `stats.json` states that no significance is claimed [ESTABLISHED]. The numbers are reported for completeness only [REASONED].

10.5 Failure taxonomy

Table 9 classifies every scored miss from `error_breakdown.json` (miss := score < 1.0 containment or < 0.5 F1; retrieval-miss := miss with no selected block overlapping gold). On RULER, SemPointer’s 123 misses split into 51 retrieval-misses and 66 block-hit-wrong cases (gold block selected, answer still wrong), with 6 gold-not-localizable rows where the gold answer cannot be localized to any chunk under the production chunking rule [ESTABLISHED]. The identical split for dense RAG (51/66/6) confirms the selection stage, not the pointer encoding, as the binding constraint [REASONED].

On LongBench the dominant categories are retrieval-miss (465) and block-hit-wrong (253), but 409 of 1150 rows are gold-not-localizable: over a third of the suite cannot reward any block selection under 512-token chunking, which compresses every system’s headroom [ESTABLISHED]. On LoCoMo, 157 of 199 rows are gold-not-localizable for SemPointer (157 for full context as well), with only 28 retrieval-misses and 14 block-hit-wrong; hit counts are 0 of 199 [ESTABLISHED]. Two structural failure modes recur across suites [REASONED]: boundary-split needles, where a gold span straddles a 512-token chunk edge so no single block contains it; and multihop splits, where the two (hotpotqa/2wikimqa) or two-to-four (musique) supporting facts land in different blocks while $m = 1$ resolves exactly one. Per-task tables serve as the hop breakdown because official hop-count labels are absent from the items (`error_breakdown.json`: design mapping hotpotqa/2wikimqa 2-hop by construction, musique 2–4-hop) [ESTABLISHED].

10.6 Missing-data disclosure

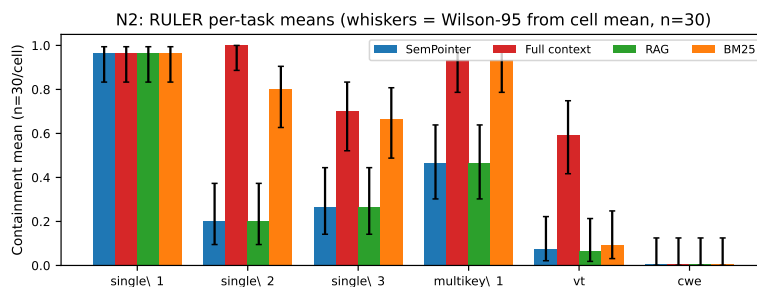
LongBench full context scored $n = 1136$ of 1150: 14 narrativeqa full-context OOMs on the 8GB card are excluded from scored means [ESTABLISHED]. Missingness strikes the longest narrativeqa contexts, so the scored-only full mean (0.1112) drops the hardest full-context cases and favors full; the errors-as-zero full mean is 0.1099, and SemPointer’s mean on exactly those 14 OOM identifiers is 0.0343, so sparse systems answered cases full context could not attempt [ESTABLISHED] (`stats.json: oom_sensitivity`). Both means are reported; the paired delta (-0.033) uses matched scored rows only [ESTABLISHED]. On RULER, 10 full-context OOM error rows were all superseded by scored retries after resume and are excluded from every analysis, with zero truly missing rows [ESTABLISHED].

10.7 Multi-hop pilot: nine arms, two clean negatives

Table 8 reports four pilots run on the same 30 musique examples (2–4-hop questions) under identical model, seed, and config, so all nine arms are matched-pair comparable; CIs overlap throughout and no significance is claimed [ESTABLISHED]. SemPointer point-estimates first (0.0541 vs 0.0468 full, 0.0313 RAG, 0.0245 BM25) at $16\times$ fewer tokens than full context (1059 vs 16925) [ESTABLISHED]. Two mechanism tests are negative. First, LLM routing from the pointer index fails twice: `agentic_pointer` (0.0299 at 3285 tokens) and the redesigned `agentic_pointer_v2` with keyword-enriched index and constrained multi-block grammar (0.0233 at 3863 tokens) both trail plain dense top-1 (0.0541 at 1059 tokens); v2 still routes a single block on 30/30 items, so the collapse is structural at this model scale, not prompt design, and no v3 is attempted [ESTABLISHED]. Second, discrete-address retrieval (`address_rag`, scoring by pointer-ID overlap only) reaches 0.0268 at 581 tokens, about 86% of dense RAG’s 0.0313: pointer IDs carry retrieval signal (k is structurally load-bearing per CPU tests) but confer no quality gain over dense cosine [ESTABLISHED]. Iterative two-round retrieval (0.0283 at 1323 tokens) matches single-round RAG, and hybrid fusion (0.0216) sits at the BM25/RAG floor [ESTABLISHED]. `session_summary` is excluded: its per-chunk summarization loop records 10–17k active tokens per request, exceeding full context, a token-accounting defect under repair rather than a result [ESTABLISHED].

Table 6. Efficiency means per system per suite (active tokens, latency, peak VRAM) with ratios vs full context computed in-script (sparse/full; <1 favors sparse).

Suite	System	Active tok.	Latency (ms)	VRAM (MB)	Tok ratio	Lat ratio	VRAM ratio
RULER	SemPointer	1270.3	894.5	2331.2	0.097	0.123	0.673
	Full context	13089.2	7256.9	3463.0	1.000	1.000	1.000
	RAG	907.2	793.7	2295.2	0.069	0.109	0.663
	BM25	893.1	765.6	2294.0	0.068	0.106	0.662
LongBench	SemPointer	963.5	1527.2	2300.5	0.074	0.189	0.768
	Full context	13066.1	8084.5	2994.4	1.000	1.000	1.000
	RAG	568.8	1228.8	2263.6	0.044	0.152	0.756
	BM25	572.7	1228.1	2263.9	0.044	0.152	0.756
LoCoMo	SemPointer	987.3	1308.8	2302.6	0.068	0.246	0.602
	Full context	14525.1	5326.2	3825.1	1.000	1.000	1.000
	RAG	571.5	1426.1	2263.8	0.039	0.268	0.592
	BM25	571.1	1425.7	2263.8	0.039	0.268	0.592

**Fig. 3.** RULER per-task containment by system ($n = 30$ cells, Wilson-95 whiskers). Easy-needle parity, hard-task separation.

10.8 Regime map: where sparse prefill wins, and where we measured

The analytic map compares per-request prefill cost of the sparse active prompt, $L_{\text{active}} = (N - m)k + mB_{\text{res}} + L_Q$, against full-context prefill over NB tokens: sparse prefill is favored where pointer overhead $(N - m)k$ stays small relative to the $NB - mB_{\text{res}}$ tokens it removes, and the advantage narrows as m grows toward N or as k approaches B [REASONED]. Figure 5 draws this map with the feasible slice—the region in which the sparse active prompt is shorter than the full prompt—labeled, and overlays the four measured operating points: RULER ($N \approx 24.75$ blocks), LongBench ($N \approx 26.7$), LoCoMo ($N \approx 28.0$), and the kv pilot ($N = 7.5$), all at $k = 8$, $m = 1$, $B = 512$ with true B of 505–510 tokens per block ($\kappa \approx 0.016$) [ESTABLISHED] (`config_table.json`). All four points sit inside the feasible slice, consistent with the measured 10–15 $\times$ active-token reductions in Table 6 [REASONED]. Contexts of 32k tokens and beyond, where the map predicts the largest prefill gaps, are untested on this 8 GB rig and remain future work; the map there is analytic projection, not measurement [REASONED].

11 What SemPointer Does, Does Not Do, and Costs

What SemPointer does (measured). On Qwen2.5-3B-Instruct (NF4, greedy, seed 42), SemPointer answers queries from a text substrate using dense per-query block selection (all-MiniLM-L6-v2, 512-token blocks), reaching mean containment 0.33 on RULER ($n=180$), mean F1 0.077 on LongBench ($n=1150$), and mean token-F1 0.052 on LoCoMo under the token-F1 adaptation ($n=199$). It reduces active tokens versus full

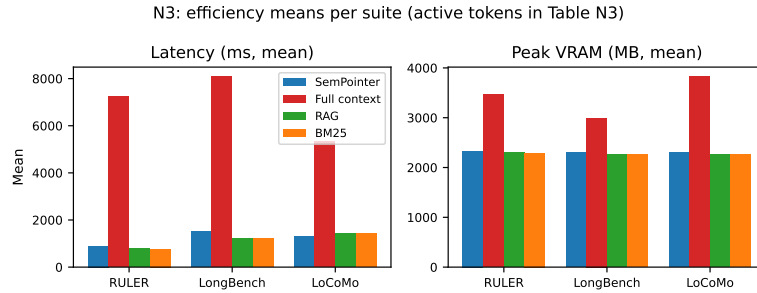


Fig. 4. Measured latency and peak VRAM per system per suite. Sparse systems dominate cost axes and trail on quality (Fig. 2).

Table 7. KV-compress pilot (n=10, containment). kv_compare n=10: pilot scale; CIs are wide and shown as-is; no significance claimed.

System	<i>n</i>	Score mean	95% CI	Active tok. / Latency (ms) / VRAM (MB)
SemPointer	10	0.8000	[0.6000, 1.0000]	725.5 / 873.1 / 2191.5
Full context	10	1.0000	[1.0000, 1.0000]	3806.4 / 1488.3 / 2513.2
KVPress SnapKV 0.3	10	0.5000	[0.2000, 0.8000]	2661.1 / 1991.9 / 4395.6
KVPress ToVA 0.3	10	0.8000	[0.5000, 1.0000]	2661.1 / 2037.2 / 4395.6
KVPress KNorm 0.3	10	0.4000	[0.1000, 0.7000]	2661.1 / 2109.7 / 4395.6

context by 10–15 $\times$ (RULER 1270 vs. 13089; LongBench 964 vs. 13066; LoCoMo 987 vs. 14525; Table N3) with correspondingly lower latency and peak VRAM on this rig. All paired deltas, 95% bootstrap CIs, and paired tests are reported as-is in Table N1 and `stats.json`.

What SemPointer does not do. Pointer tokens do not affect retrieval: SemPointer and the dense-RAG arm select identical blocks (RULER retrieval-miss counts 51 vs. 51; Table N5), so the measured RULER delta versus RAG is +0.001 (95% CI $[-0.003, +0.006]$, McNemar $p = 1.000$). There is no KV-substrate end-to-end result: the KV comparison is a detached n=10 pilot (Table N4), and no pointer is ever resolved into a live KV cache during generation. There is no k -sweep: pointer length k is fixed in these runs, so no claim about k sensitivity is supported. There is no multi-model evidence: every number comes from the single Qwen2.5-3B-NF4 configuration.

What SemPointer costs. Each query pays a two-call routing overhead (one dense-selection pass plus one generation pass over the resolved prompt). SemPointer consumes roughly 1.4–1.7 $\times$ the active tokens of dense RAG (1270 vs. 907 on RULER; 964 vs. 569 on LongBench; 987 vs. 572 on LoCoMo) for statistically indistinguishable quality (LongBench delta -0.0009 , 95% CI $[-0.0047, +0.0028]$, Wilcoxon $p = 0.32$; LoCoMo delta $+0.0017$, 95% CI $[-0.005, +0.008]$, $p = 0.69$). Against full context it trades quality for tokens: RULER -0.37 ($[-0.44, -0.30]$, McNemar $p = 6.7 \times 10^{-14}$), LongBench -0.033 ($[-0.040, -0.027]$, Wilcoxon $p = 1.6 \times 10^{-31}$), LoCoMo -0.012 ($[-0.022, -0.003]$, Wilcoxon $p = 0.028$, not significant at the Bonferroni-adjusted $\alpha = 0.0038$ over 13 tests). BM25 exceeds SemPointer on RULER by +0.248 in full-context-normalized terms (SemPointer-vs-BM25 delta -0.248 , $[-0.314, -0.183]$), driven by variable-tracking and multikey tasks (Table N2).

Table 8. Multi-hop pilot (LongBench musique, $n=30$ matched examples, Qwen2.5-3B-NF4, greedy, seed 42). all four pilots ran the same 30 examples under identical config, so all nine arms are directly comparable. CIs are wide at $n=30$; no significance is claimed. session summary excluded (token-accounting defect, see text).

System	n	F1 mean	95% CI	Active tok	Latency (ms)
sempointer	30	0.0541	[0.0207, 0.1007]	1059	1461
full	30	0.0468	[0.0296, 0.0652]	16925	9630
rag	30	0.0313	[0.0160, 0.0491]	571	1446
bm25	30	0.0245	[0.0117, 0.0385]	581	1340
agentic pointer	30	0.0299	[0.0146, 0.0473]	3285	3386
agentic pointer v2	30	0.0233	[0.0083, 0.0416]	3863	2812
iterative rag	30	0.0283	[0.0146, 0.0436]	1323	2871
hybrid	30	0.0216	[0.0095, 0.0368]	581	1389
address rag	30	0.0268	[0.0124, 0.0438]	581	1536

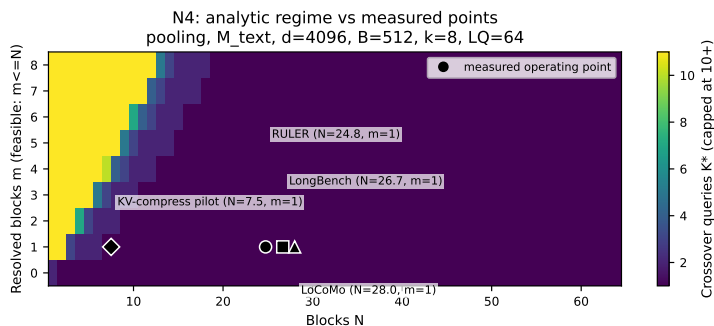


Fig. 5. Analytic prefill-cost regime map over registry size N and resolved blocks m , with measured operating points overlaid (feasible slice only).

12 Discussion and Open Challenges

A. Differentiable vs. Discrete Addressing: Discrete tokens are cache-compatible and supported by standard tokenization pipelines. However, they impose a quantization bottleneck: the pointer space is finite, and gradients cannot flow through discrete selection without additional estimators such as Gumbel-softmax or REINFORCE. Continuous soft pointers avoid this restriction and offer flexible expressive capacity, but they introduce opacity and susceptibility to positional drift across transformer layers. In our measurements, selection is performed by dense cosine over block embeddings (pointer strings are prompt-resident only; see Section 9), so the addressing choice does not affect retrieval quality in these runs. Neither option is clearly superior; the choice depends on the implementation of $\mathcal{G}_\phi$ and the resolution operator.

B. Dynamic Memory Granularity: Fixed chunking produces clean algebraic guarantees. In multi-turn dialogue, proposition boundaries vary dynamically. Integrating learned segmentation heads into $\mathcal{G}_\phi$ to identify variable-length semantic units remains an important architectural direction.

C. Safety and Indirection Attacks: Decoupling pointers from storage creates distinct security considerations. Adversarial inputs designed to collide with existing addresses could trigger unauthorized memory resolution. Access control lists, pointer provenance tracking, and cryptographic address hashing are natural design responses to be evaluated in future work.

Table 9. Miss taxonomy counts per suite (from `error_breakdown.json`). Miss := score<1.0 (containment) or score<0.5 (F1, lenient); retrieval-miss := miss with selected_ids $\cap$ gold_blocks = $\emptyset$. **block-hit-wrong:** gold block selected but answer wrong; **unlocalizable:** gold answer not localizable to any chunk under the production chunking rule; **error-row:** scored-row errors (`n_errors` in `metrics.json`). Full-context misses carry no retrieval attribution (NA).

Suite	System	n	Hit	Miss	Retrieval-miss	Block-hit-wrong	NA	Unlocalizable	Error-row
RULER	SemPointer	180	57	123	51	66		6	0
	Full context	180	117	63	0	60 (NA)		3	0
	RAG	180	57	123	51	66		6	0
LongBench	BM25	180	101	79	19	54		6	0
	SemPointer	1150	23	1127	465	253		409	0
	Full context	1136	29	1107	0	717 (NA)		390	14
	RAG	1150	20	1130	465	254		411	0
LoCoMo	BM25	1150	23	1127	458	261		408	0
	SemPointer	199	0	199	28	14		157	0
	Full context	199	0	199	0	42 (NA)		157	0
	RAG	199	0	199	28	14		157	0
KV-compress pilot (pilot n=10)	BM25	199	1	198	15	27		156	0
	SemPointer	10	8	2	2	0		0	0
	Full context	10	10	0	0	0		0	0
	KVPress SnapKV 0.3	10	5	5	0	5 (NA)		0	0
	KVPress ToVA 0.3	10	8	2	0	2 (NA)		0	0
	KVPress KNorm 0.3	10	4	6	0	6 (NA)		0	0

13 Limitations

M1: single model. Every measurement uses Qwen2.5-3B-Instruct (NF4, greedy, eager attention); no other model family or scale was run (BENCHMARK_PROVENANCE.md §4). Cross-model statements are unsupported.

M2: single seed. All runs use seed 42 with no seed sweep (`stats.json: "seed": 42`). Reported CIs are bootstrap resampling intervals over examples, not intervals over training or sampling seeds.

M3: no fine-tuning. Neither the reader model nor the MiniLM selection encoder was trained or adapted for pointers; results describe an inference-only assembly of frozen components.

M4: 16k context cap. Nominal haystack budgets are 4096/8192/16384 tokens (BENCHMARK_PROVENANCE.md §1); nothing longer was evaluated, and realized lengths are word-estimated within ~20% of nominal.

M5: KV evidence is a pilot. The KV comparison is $n=10$ with wide CIs (e.g. SemPointer-vs-full -0.20 , $[-0.50, 0.00]$, McNemar $p = 0.480$; `stats.json final_kvcompare`, Table N4). The `pilot_note` field states no significance is claimed.

M6: text substrate only. All resolution paths return verbatim text; the KV-substrate dereference was never executed end-to-end (INDEPENDENT_RECONSTRUCTION.md §4). Cached-regime memory claims remain analytic, not measured.

M7: pointer ablation is approximately zero. SemPointer and dense RAG produce identical selection outcomes (RULER retrieval-miss 51/51; LongBench 465/465; Table N5), so these runs cannot attribute any quality effect to the pointer strings themselves.

M8: no k -sweep. Pointer length and top- m selection depth are fixed; sensitivity of quality or cost to k was not measured in this study.

M9: OOM asymmetry favors full context. Fourteen narrativeqa full-context rows OOMed and are excluded from scored means; scored-only reporting drops the hardest full-context cases (full mean 0.1112 scored-only vs. 0.1099 errors-as-zero; `stats.json oom_sensitivity`, Table N1 caption).

M10: LoCoMo uses a metric adaptation. With no judge API available, all LoCoMo categories are scored with plain token F1 instead of the official GPT-4 judge plus stemming protocol, so LoCoMo numbers are not comparable to published values (BENCHMARK_PROVENANCE.md §3; every table labels this adaptation).

M11: latency and VRAM are rig-specific. Latency and peak-VRAM means (Table N3) were recorded on a single 8 GB laptop GPU path and are reported without CIs; they describe this setup, not serving hardware generally.

M12: per-task uncertainty is partial. Suite-level bootstrap CIs are stored in `metrics.json`; per-task RULER cells (Table N2) carry Wilson-95 whiskers computed in-script from cell means only (Fig. N2), not paired tests.

M13: pending arms. LongMemEval [32] evaluation and a no-context control arm are not run here; LongMemEval is assessed as future work requiring a judge API and 115k-token histories beyond this rig (BENCHMARK_PROVENANCE.md §5). TTFT/decode logging, a BGE encoder swap [33]—the C-Pack packed-resources embedding family released with C-MTEB/C-MTP, with E5 [30] as the weakly-supervised contrastive-pretraining encoder family—which selects different blocks than MiniLM on a probe, and an LLMingua scaffold are implemented and smoke-verified; reranker and compression arms are not yet run at benchmark scale.

14 Conclusion

Measured on Qwen2.5-3B-NF4 (seed 42) across RULER, LongBench, and LoCoMo, SemPointer exchanges 10–15× fewer active tokens for lower quality than full context (RULER −0.37, LongBench −0.033, LoCoMo −0.012, the last not significant after Bonferroni correction), while matching dense RAG quality at 1.4–1.7× the token cost. The failure taxonomy (Table N5) shows selection misses dominate on RULER and unlocalizable answers dominate on LoCoMo, and the identical selection outcomes of the SemPointer and RAG arms leave the pointer strings with no measured quality contribution in these runs. The mechanism verdict is therefore narrow: dense per-query routing over a text substrate works as an assembly; the multi-hop pilot (Table N6) returns two clean negatives, LLM-routing-from-index (twice, including a redesigned router that still single-blocks 30/30 items) and discrete-address retrieval at 86% of dense with no gain; pointer-mediated addressing as distinct from standard retrieve-then-read remains unproven pending the KV end-to-end path. The future program is fixed: full reruns at $n \geq 500$ with multiple seeds, 32k-plus contexts on larger hardware, a measured pointer-length sweep, pointer-token scoring that separates address effects from selection effects, a recomputed KV comparison at pilot scale or above, and LongMemEval oracle-setting evaluation once a judge is available.

Data and Code Availability: All measurements derive from executed benchmark runs under `experiments/results/final_*/` (RULER $n=180$, LongBench $n=1150$, LoCoMo $n=199$, kv-compress pilot $n=10$) with configs, environment records, and per-example predictions; generator scripts (`stats_real.py`, `error_taxonomy.py`, `make_paper_artifacts.py`) reproduce every table and figure. Runners and code are available at <https://github.com/VIKSIT-GARG/Sempointer>. **Funding / Ethics:** No external funding; all evaluations utilized public academic benchmarks without private data.

A Retired Numbers

Prior drafts reported synthetic-harness figures (E01–E17 series) that forensic re-analysis withdrew; the full mapping of each retired number to its disposition is retained under `sections/_contradiction_manifest.tex` in the repository and supplied as supplementary material.

References

1. Blouw, P., Solodkin, E., Thagard, P., Eliasmith, C.: Concepts as Semantic Pointers: A Framework and Computational Model. *Cognitive Science*, vol. 40, pp. 1128–1162 (2016).
2. Cheng, X., Wang, X., Zhang, X., Ge, T., Chen, S., Wei, F., Zhang, H., Zhao, D.: xRAG: Extreme Context Compression for Retrieval-augmented Generation with One Token. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37 (2024).
3. Chevalier, A., Wettig, A., Ajith, A., Chen, D.: Adapting Language Models to Compress Contexts. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3829–3846 (2023).
4. Eliasmith, C.: *How to Build a Mind: Toward a Comprehensive Theory of Cognition*. Oxford University Press (2013).
5. Gao, J., Lv, Q., Wang, Z., Wu, T., Cao, Z., Li, W.: UniICL: An Efficient ICL Framework Unifying Compression, Selection, and Generation. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 24–40 (2025).
6. Ge, T., Hu, J., Wang, X., Chen, S., Wei, F.: In-context Autoencoder for Context Compression in a Large Language Model. In: *International Conference on Learning Representations (ICLR)* (2024).
7. Gim, I., Chen, G., Lee, S., Sarda, N., Khandelwal, A., Zhong, L.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In: *Proceedings of Machine Learning and Systems (MLSys)* (2024).
8. Jiang, H., Wu, Q., Lin, C., Yang, Y., Qiu, L.: LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 13358–13376 (2023).
9. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C., Yang, Y., Qiu, L.: LongLLMingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 91–107 (2024).
10. Kim, J., Yeom, J., Yun, S., Song, H.: Compressed Context Memory For Online Language Model Interaction. In: *International Conference on Learning Representations (ICLR)* (2024).
11. Kocisky, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K.M., Melis, G., Grefenstette, E.: The NarrativeQA Reading Comprehension Challenge. *Transactions of the Association for Computational Linguistics (TACL)*, vol. 6, pp. 317–328 (2018).
12. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient Memory Management for Large Language Model Serving with PagedAttention. In: *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 611–626 (2023).
13. Lajewska, W., Hardalov, M., Aina, L., John, N.A., Su, H., Marquez, L.: Understanding and Improving Information Preservation in Prompt Compression for LLMs. In: *Findings of the Association for Computational Linguistics: EMNLP 2025* (2025).
14. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474 (2020).

15. Li, Y., Su, D., Collier, N.: 500xCompressor: Generalized Prompt Compression for Large Language Models. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 1219–1235 (2025).
16. Liu, D., Chen, M., Lu, B., Jiang, H., Han, Z., Zhang, Q., Chen, Q., Zhang, C., Ding, B., Zhang, K., Chen, C., Yang, F., Yang, Y., Qiu, L.: RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In: Advances in Neural Information Processing Systems (NeurIPS) (2025).
17. Liu, X., Zhao, R., Huang, P., Liu, X., Xiao, J., Xiao, C., Xiao, T., Gao, S., Yu, Z., Zhu, J.: Autoencoding-Free Context Compression for LLMs via Contextual Semantic Anchors. arXiv preprint arXiv:2510.08907 (2025).
18. Luo, X., Wang, Y., Zhu, Q., Zhang, Z., Zhang, X., Yang, Q., Xu, D., Che, W.: Turning Trash into Treasure: Accelerating Inference of Large Language Models with Token Recycling. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 338–354 (2025).
19. Mohtashami, A., Jaggi, M.: Landmark Attention: Random-Access Infinite Context Length for Transformers. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36 (2023).
20. Mu, J., Li, X.L., Goodman, N.: Learning to Compress Prompts with Gist Tokens. Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 48356–48374 (2023).
21. Munkhdalai, T., Faruqui, M., Gopal, S.: Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. arXiv preprint arXiv:2404.07143 (2024).
22. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E.: MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560 (2023).
23. Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Ruhle, V., Yang, Y., Lin, C., Zhao, H.V., Qiu, L., Zhang, D.: LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In: Findings of the Association for Computational Linguistics (ACL Findings), pp. 57–74 (2024).
24. Rau, D., Wang, S., Dejean, H., Clinchant, S., Kamps, J.: Context Embeddings for Efficient Answer Generation in Retrieval-Augmented Generation. In: Proceedings of the 18th ACM International Conference on Web Search and Data Mining (WSDM), pp. 112–121 (2025).
25. Tack, J., Kim, J., Mitchell, E., Shin, J., Teh, Y.W., Schwarz, J.R.: Online Adaptation of Language Models with a Memory of Amortized Contexts. arXiv preprint arXiv:2403.04317 (2024).
26. Tan, S., Li, X., Patil, S.G., Wu, Z., Zhang, T., Keutzer, K., Gonzalez, J.E., Popa, R.A.: LLoCO: Learning Long Contexts Offline. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 975–991 (2024).
27. Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., Han, S.: Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In: Proceedings of the International Conference on Machine Learning (ICML) (2024).
28. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: MuSiQue: Multihop Questions via Single-hop Question Inversion. Transactions of the Association for Computational Linguistics (TACL), vol. 10, pp. 539–554 (2022).
29. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008 (2017).

30. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., Wei, F.: Text Embeddings by Weakly-Supervised Contrastive Pre-training. arXiv preprint arXiv:2212.03533 (2022).
31. Wu, Y., Rabe, M.N., Hutchins, D., Szegedy, C.: Memorizing Transformers. In: International Conference on Learning Representations (ICLR) (2022).
32. Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K., Yu, D.: LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv preprint arXiv:2410.10813 (2024).
33. Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., Nie, J.: C-Pack: Packed Resources For General Chinese Embeddings. arXiv preprint arXiv:2309.07597 (2023).
34. Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Sun, M.: InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024).
35. Xu, F., Shi, W., Choi, E.: RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. In: International Conference on Learning Representations (ICLR) (2024).
36. Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W.W., Salakhutdinov, R., Manning, C.D.: HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 2369–2380 (2018).
37. Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., Jiang, J.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In: Proceedings of the Twentieth European Conference on Computer Systems (EuroSys) (2025).